

구아름 협업일지 요약

중급프로젝트 RFP RAG · 기록 기간 2026-07-13 ~ 2026-07-31

역할 요약

구아름 님은 프로젝트가 팀 단위로 실행될 수 있도록 GCP VM·SSH·GitHub 개발환경을 구축하고, RAG의 후반부인 답변 생성, 비동기 처리, 멀티턴 대화, 문맥 압축, RAGAS-LLM Judge 평가를 담당했다. 개인 PC에만 머무르지 않도록 공용 VM과 `/home/data` 권한을 정리했으며, 검색 담당자가 반환한 metadata를 답변과 근거 문서에 연결했다. 후반에는 생성 결과를 평가하는 코드와 README를 추가해 개발환경에서 최종 평가·제출 단계까지 이어지게 했다.

핵심 기여

1. GCP VM과 공동 개발환경 구축

- 팀 프로젝트용 GCP VM을 생성하고 GPU region·x86 환경 등 VM 생성 시 확인할 조건을 정리했다.
- 팀원들이 VS Code에서 VM으로 바로 접속할 수 있도록 SSH 사용자 설정과 Remote SSH 사용 절차를 문서화했다.
- GitHub 저장소의 기본 경로, webhook, lint 검사, repository 규칙과 pre-commit 설정을 구성했다.
- 팀원 계정의 SSH 접속 문제를 함께 점검하고, 개인 키 또는 로컬 환경 때문에 생기는 문제를 감사에게 확인했다.
- `/home/data`를 팀원이 함께 사용할 수 있도록 접근 권한을 설정하고, uv 설치·동기화 절차를 README에 정리했다.

이 작업으로 팀원들은 각자 계정과 브랜치에서 코드를 개발하면서도 원본 데이터, 청크, Chroma DB와 실행 결과는 공용 GCP 경로에서 공유할 수 있게 됐다.

2. Naive 답변 생성과 로깅

- Naive RAG용 Generation 코드를 작성해 검색 결과를 LLM 답변으로 연결했다.
- 응답 완료시간, 검색·LLM 구간, 대화 기록을 확인할 수 있도록 logger를 추가했다.
- 답변과 함께 근거 문서, `chunk_id`, `table_id`, 신뢰도 수치를 출력하도록 결과 형식을 확장했다.
- logger와 화면 출력이 중복되는 문제를 수정해 사용자 출력과 운영 로그가 섞이지 않도록 정리했다.
- Naive 코드를 별도 브랜치에 백업해 Advanced 개발 과정에서도 기준선을 잃지 않도록 관리했다.

3. 비동기 처리와 멀티턴 대화 설계

- 동기식 답변 생성을 비동기식으로 바꿔 검색·생성 흐름을 확장할 수 있도록 했다.
- 후속 질문이 이전 대화에 의존할 때 독립적인 검색 질의로 바꾸는 query rewriting을 적용했다.
- 멀티턴 기억을 단순히 전체 대화를 계속 붙이는 방식이 아니라 다음 세 층으로 설계했다.

기억 계층	역할
최근 대화	최근 4~6턴을 원문으로 유지해 직전 맥락 반영
구조화된 세션 상태	사업명, 기관명, 예산, 기간, 마감일 등 핵심 필드 누적
장기 기억	오래된 대화를 요약하거나 필요한 과거 질의를 다시 검색

- 검색 문서는 점수가 낮거나 중복된 결과를 제거하고 최대 개수를 제한했으며, 문맥 길이가 과도하게 늘면 압축하는 토큰 예산 관리 방식을 적용했다.
- `project_name`, `organization`, `budget`, `duration`, `deadline`, `eligibility`, `evaluation_criteria`, `required_documents` 등 RFP Q&A에 필요한 필드를 세션 상태로 관리하도록 정리했다.

이 구조는 대화 전체를 무조건 프롬프트에 넣지 않고, RFP 문서에서 이미 확인한 핵심 정보를 유지하면서 토큰 사용량을 줄이기 위한 설계였다.

4. Retriever metadata와 Generation 연동

- Retriever가 반환하는 metadata 형식에 맞춰 Generation 코드를 수정했다.
- 답변에서 사용한 근거 문서와 체크 식별자를 보여 주도록 해, 사용자가 어떤 문서에 근거한 답변인지 확인할 수 있게 했다.
- Golden Set 형식을 팀과 논의하면서 생성 답변과 검색 평가가 같은 질문·근거 기준을 사용하도록 조율했다.
- VM 디스크가 98%까지 찬 상황과 멀티턴 실행 중 GPU 메모리 부족 문제를 기록하고 팀에 공유했다.

5. RAGAS-LLM Judge 평가

- RAGAS 평가 코드와 실행 설명서를 작성했다.
- 평가 결과를 JSON으로 저장하는 과정의 오류를 해결하고, 팀에서 받은 test set으로 실행하도록 연결했다.
- RAGAS metric이 출력되지 않는 문제를 팀에 공유하고 도움을 받아 해결했다.
- OpenAI API 요청 중 429 Too Many Requests가 발생했을 때 API 제한 문제를 확인했다.
- 자동 생성된 질문·정답이 실제로 옳은지 검증해야 한다는 문제를 제기하고, RAGAS 외에 AI 모델 기반 LLM Judge 코드를 추가했다.
- 마지막으로 평가·실행 방법을 README에 반영해 팀원이 같은 절차를 재현할 수 있도록 했다.

기간별 작업 흐름

기간	주요 작업	협업 결과
7/13~7/15	GCP VM, VS Code SSH, GitHub 규칙 ·lint:pre-commit 구성	팀 공동 개발환경과 접속 문서 마련
7/20~7/22	<code>/home/data</code> 권한, <code>uv</code> 문서, Naive Generation-logger, 코드 백업	검색 결과를 답변으로 연결하는 기준선 확보
7/23~7/24	비동기 처리, query rewrite, 멀티턴, 문맥 압축, 토큰 예산	RFP형 멀티턴 답변 구조 설계
7/27~7/31	RAGAS, test set, JSON 오류 수정, LLM Judge, README	생성 결과 평가와 재현 절차 정리

문제 해결 기록

문제	대응	의미
팀원별 SSH 접속 차이	사용자·키·로컬 환경을 분리해 점검하고 설정 문서 작성	공용 VM 진입 장벽 완화
답변이 이전 대화를 잘 기억하지 못함	최근 대화·요약·구조화 필드를 분리한 멀티턴 상태 설계	문맥 유지와 토큰 절약을 함께 고려
logger와 화면 출력 중복	출력 경로를 정리	사용자 화면과 운영 기록 분리
멀티턴 GPU 메모리 부족	문제를 공용 이슈로 공유하고 실행 환경 점검	팀 자원 사용 상태를 함께 관리
RAGAS metric 미출력	팀에 도움을 요청해 원인 수정	평가 코드가 실제 수치를 내도록 복구
API 429 오류	rate limit 문제로 구분	데이터·코드 오류와 외부 API 제한 분리
생성 문답의 정답 신뢰성	LLM Judge 추가, Golden Set 양식 논의	자동 평가만 믿지 않고 평가 데이터 품질 확인

협업에서 드러난 강점

- 1 팀이 같은 환경에서 일하도록 기반을 먼저 만들었다. VM 생성부터 SSH, GitHub 규칙, 공용 데이터 권한까지 개발 전제조건을 정리했다.
- 2 생성 기능을 대화형 서비스 수준으로 확장했다. 단일 질문·답변을 넘어 비동기 처리, 질의 재작성, 세션 상태와 토큰 예산을 함께 설계했다.
- 3 검색 결과의 근거를 사용자에게 노출했다. 근거 문서와 청크 ID, 신뢰도를 답변에 연결해 추적 가능성을 높였다.
- 4 평가 실패를 숨기지 않고 공유했다. metric 누락, API 429, GPU·디스크 문제를 팀 이슈로 드러내고 평가 흐름을 보완했다.